

Prompt de implementação — melhorias prioritárias do SISPNAIST

Copie o texto abaixo para o agente de código que terá acesso ao repositório. Revise o plano antes de autorizar ações que mudem permissões ou dados.

Você é uma pessoa engenheira sênior de segurança de aplicações e full-stack. Trabalhe no repositório SISPNAIST que já está aberto no workspace. A aplicação usa React + TypeScript + Vite no frontend, Node.js + Express + TypeScript no backend, MongoDB Atlas e deploy por Vercel/Render.

Leia primeiro o relatório `Doc/relatorio_auditoria_tecnica_2026-10-01.md` e confirme os achados diretamente no código atual; não presuma que o relatório corresponde a uma branch diferente. O objetivo é corrigir defeitos reproduzíveis, mantendo a compatibilidade e sem alterar dados ou serviços de produção.

Regras obrigatórias

1. **Não publique nada.** Não acione deploy, webhook, migração, seed, alteração do Atlas, configuração do Render/Vercel, e-mail real, compra ou integração externa. Não rode comandos destrutivos nem altere registros reais. Deixe qualquer deploy para aprovação humana.
2. **Segredos e dados pessoais:** não leia, imprima, copie, adicione ao diff ou inclua em logs/relatórios os valores de `backend/.env`, `backend/admin.txt`, credenciais, tokens, CPFs, endereços, laudos ou dados de produção. Trate esses arquivos como sensíveis; reporte apenas nomes de arquivos/variáveis. Não faça rotação de chaves; proponha os passos manuais para o responsável.
3. **Não faça atualização cega de dependências.** Não use `npm audit fix --force`. Confira advisories oficiais atuais e compatibilidade. Mudanças major devem ficar isoladas e explicadas.
4. **Permissões são decisão de produto.** Se a política exata para os perfis `admin`, `gestor`, `saude` e `trabalhador` não estiver documentada, adote negação por padrão para dados fora do escopo e registre a matriz assumida para validação humana. Nunca amplie acesso para fazer os testes passarem.
5. Faça mudanças pequenas, com teste que demonstre a falha antes e o bloqueio depois. Preserve o comportamento legítimo do fluxo; não reescreva módulos sem necessidade.

Etapa 1 — diagnóstico sem alteração

- Mostre branch, estado inicial do Git e versão Node/npm sem exibir conteúdo de `.env` ou arquivos privados.
- Confirme os achados no backend, frontend, lockfiles e CI; liste arquivos e funções relevantes.
- Apresente uma matriz proposta de permissões e uma sequência de mudanças antes de editar. Marque toda decisão de produto pendente.
- Verifique `npm audit` para cada pacote e fontes oficiais; registre pacotes/faixas, sem copiar segredos nem dados pessoais.

Etapa 2 — correções de segurança prioritárias

Implemente primeiro estes itens, em commits/diffs lógicos independentes:

1. **Autorização multiempresa fail-closed.** Em `backend/src/utils/scope.ts`, elimine consulta sem filtro quando o perfil `gestor` não tiver empresa válida. Retorne erro de autorização ou filtro impossível. Atualize o teste que atualmente espera `{}` para esse caso.
2. **Escritas de registros de saúde.** Revise `routes/acidentes.ts`, `routes/doencas.ts`, `routes/submodulosTrabalhador.ts`, `routes/trabalhadores.ts`, controllers correspondentes e endpoints correlatos. Antes de criar, alterar, excluir, listar ou exportar, valide perfil e empresa/unidade do trabalhador-alvo no servidor. A verificação deve integrar o filtro da consulta MongoDB; não confie em ID vindo do corpo nem só no frontend. Um gestor não pode criar para outra empresa; perfil `saude` não pode ignorar escopo; a pessoa trabalhadora não pode editar o próprio histórico sem permissão explícita.
3. **Vínculo do item pai.** Ao atualizar/excluir informação/submódulo/registro por ID, primeiro busque o registro dentro do trabalhador e empresa autorizados; não atualize por `infolist` isolado e não permita trocar `trabalhadorid` sem revalidar o novo alvo. Use operação atômica/condição de escopo para evitar TOCTOU.
4. **Uploads.** Restrinja listagem de metadados e downloads ao tenant autorizado, independentemente de filtro enviado pelo cliente. Atualize Multer para a versão estável atualmente corrigida — consulte advisory OpenJS/Express vigente; o relatório consultou 2.3.0 como patch de agosto de 2026. Configure limites multipart, incluindo `fieldArrayIndexLimit` conforme a necessidade da aplicação. Adapte middleware/typing se necessário e adicione testes contra multipart malformado.
5. **Token de verificação de e-mail.** Faça DTO explícito para respostas de usuário. `verificationToken`, expiração, refresh tokens, senha/hash, códigos 2FA e histórico de senhas jamais podem ser serializados. Adicione teste que verifique a ausência desses campos no cadastro e exija verificação real pelo link de e-mail.

6. **JWT.** Configure access token de curta duração (15m como valor inicial coerente com .env.example) e segredo de refresh independente. Em produção, rejeite startup se faltar JWT_REFRESH_SECRET ; remova fallback para compartilhar JWT_SECRET . Teste tipo de token, expiração, refresh, logout e revogação. Não migre a sessão do usuário para outro mecanismo sem explicar o impacto e testes.
7. **JWT em URL.** Remova aceitação de JWT de acesso via query string em authMiddleware ; use somente o mecanismo de autenticação aprovado. Preserve links assinados de download apenas se forem tokens separados, limitados ao recurso, com expiração curta, proteção contra log/referrer e validação de escopo.
8. **Auditoria minimizada.** Nunca grave documento médico completo. Registre evento mínimo e IDs; aplique lista permitida de campos e mascaramento recursivo de CPF/e-mail/telefone/endereço/campos clínicos/tokens. Não silencie falha de auditoria em operações que o produto exige rastrear; se a operação não puder falhar por audit outage, registre claramente o risco/alarme. Não registre corpo ou tokens em logs.

Etapa 3 — disponibilidade e configuração

- Faça a readiness depender da conexão real do Atlas. GET /health não deve responder pronto quando banco/dependências obrigatórias estão indisponíveis. Separe liveness e readiness; aguarde conexão e inicialização crítica, com encerramento/retry limitado quando Atlas não conecta.
- Não troque o armazenamento de uploads nem migre arquivos sem escolha do responsável. Produza uma interface de storage e uma proposta de migração para object storage privado; considere que disco local gratuito do Render é efêmero. Nenhuma migração de produção nesta tarefa.
- Verifique configuração requerida no Render em documentação/código, sem consultar ou exibir valores secretos. Documente FRONTEND_URL , provedor de e-mail, JWT_REFRESH_SECRET e demais variáveis necessárias. Não introduza URLs localhost como fallback de produção.
- Defina readiness operacional e runbook de restore/backup sem acessar nem modificar o Atlas.

Etapa 4 — dependências, frontend e CI

- Trate advisories de produção primeiro; em especial Axios/Joi/Nodemailer no backend e Axios/PostCSS/Vite/React Router no frontend conforme o lockfile atual. Faça uma atualização por grupo pequeno, rode testes e revise breaking changes. Para Vite major, mantenha a migração separada.

- Corrija os erros TypeScript do frontend no modelo de dados/contratos da API (IDs/populates, tipos ausentes e propriedades divergentes). Não use `any`, `@ts-ignore`, nem desligue `strict` como atalho.
- Decida/teste o runner para `frontend/src/__tests__/masks.test.ts` e a dependência `@jest/globals`; configure lint de forma explícita para os scripts existentes.
- Faça CI exigir type-check, lint, testes e build. Alinhe Node de CI com a versão suportada escolhida para deploy. Trivy/secret scanning ou equivalente deve efetivamente bloquear findings relevantes; não marque o scan de código como `continue-on-error` sem justificativa.

Testes de segurança mínimos obrigatórios

Use banco de teste/mock isolado, sem Atlas de produção. Para cada operação sensível, crie usuário de empresa A e trabalhador de empresa B. Prove que:

- gestor A não lista, lê, cria, edita, exclui ou exporta dados de B;
- usuário `gestor` sem empresa recebe 403 ou resultado impossível, nunca consulta global;
- `saude` fica limitado à matriz aprovada e não contorna o controle só por não ser `trabalhador`;
- `trabalhadorId` do body/query não altera o escopo;
- `infolist / itemId` de outro trabalhador não permite leitura, alteração, reatribuição ou exclusão;
- metadados/links de upload não atravessam empresas;
- token de verificação/refresh/códigos/senha não aparecem em resposta JSON nem logs;
- tokens access/refresh/2FA não são intercambiáveis, expiram no prazo e refresh não prolonga indevidamente access;
- requests multipart malformados retornam erro controlado sem derrubar processo e respeitam limites;
- readiness responde não pronto sem banco;
- logs de auditoria não carregam PII/relato clínico integral.

CrITÉRIOS para concluir

1. Apresente um diff curto e organizado por achado; mantenha o repositório livre de segredos e dados reais.
2. `npm run build` do backend e os testes do backend passam sem `--forceExit` ou com a

razão dos handles corrigida. Execute também lint.

3. Frontend `npm run type-check` , testes configurados, lint e `npm run build` passam. Não deixe erros TypeScript conhecidos.
4. `npm audit` foi repetido nos dois pacotes. Não há vulnerabilidade alta aplicável sem correção ou exceção documentada, mitigação, owner e prazo.
5. CI reproduz os mesmos comandos locais e falha de modo seguro. Nenhum deploy foi acionado.
6. Entregue resumo por achado, arquivos alterados, testes/comandos e resultados, decisões pendentes de produto e passos manuais seguros para rotação de credenciais/configuração externa.

Comece pela Etapa 1, sem editar arquivos. Pare e peça decisão se descobrir que corrigir a autorização exige uma escolha de produto não inferível do código; até essa decisão, mantenha a negação por padrão.
